UNIVERSIDAD DE MURCIA

Universidad Politécnica de Cartagena

Trabajo Final

Clasificación de piezas mediante Deep Learning

Análisis de Imagen y Visión Artificial

Curso: 2025/2026

Rubén José Tenza Martín

1. Introducción....2
1.1. Contexto y Justificación....2
1.2. Definición del Problema....2
1.3. Objetivos del Proyecto....2
1.4. Estructura de la Memoria....3
2. Dimensionamiento del Sistema de Captura (Hardware)....3
2.1. Cálculo de la Resolución Espacial....3
2.2. Selección de la Cámara....3
2.3. Cálculo de la Óptica (Distancia Focal)....4
2.4. Análisis del Tiempo de Exposición (Motion Blur)....4
2.5. Presupuesto Aproximado del Equipo....4
3. Desarrollo del Software y Metodología Experimental....5
3.1. Fase de Preparación y Curación del Dataset....5
3.1.1. Extracción y Estructuración (Celdas 1 y 2)....5
3.1.2. Análisis Estadístico y Balanceo (Celda 3)....7
3.2. Ingeniería de Características y Preprocesamiento (Celda 4)....7
3.2.1. Aplicación de CLAHE (Contrast Limited Adaptive Histogram Equalization)....7
3.2.2. Normalización de Rango....8
3.3. Configuración de los Generadores de Datos....8
3.3.1. Aumento de Datos (Data Augmentation)....8
3.4. Arquitectura del Modelo: MobileNetV2 y Cabezal de Alta Precisión....9
3.4.1. Justificación de MobileNetV2....9
3.4.2. El Cabezal de Clasificación Personalizado (Top Layers)....9
3.4.3. Estrategias contra el Sobreajuste (Overfitting)....10
3.4.4. Proceso de Entrenamiento en Dos Fases....10
4. Análisis de Resultados y Evaluación del Rendimiento....12
4.1. Análisis de las Curvas de Aprendizaje....12
4.2. Métricas de Clasificación Detalladas....13
4.3. Interpretación de la Matriz de Confusión....14
4.4. Rendimiento en Tiempo Real....15
5. Conclusiones y Trabajos Futuros....16
5.1. Cumplimiento de Objetivos....16
5.2. Robustez y Escalabilidad....16
6. Bibliografía y Referencias....17
6.1. Fuentes Académicas y Dataset....17
6.2. Hardware y Visión Industrial (Precios y Especificaciones)....17
6.3. Software y Documentación Técnica....17

1. Introducción

1.1. Contexto y Justificación

En el marco de la **Industria 4.0**, la automatización de los procesos de control de calidad se ha convertido en un pilar fundamental para garantizar la eficiencia y la competitividad. Dentro de este ecosistema, la **Visión Artificial** destaca como una disciplina capaz de emular la capacidad visual humana para obtener, procesar y analizar información de forma masiva y precisa.

Históricamente, la clasificación de piezas mecánicas en entornos industriales dependía de algoritmos de visión clásica basados en descriptores geométricos fijos. Sin embargo, la llegada del **Deep Learning** ha supuesto una disrupción técnica, permitiendo que sistemas basados en Redes Neuronales Convolucionales (CNN) aprendan características abstractas y complejas de los objetos, ofreciendo una robustez superior ante variaciones de iluminación, oclusiones parciales y cambios en la orientación de las piezas.

1.2. Definición del Problema

El presente trabajo final, denominado **TF02: Clasificación de piezas mediante Deep-learning**, aborda un reto real de inspección técnica en una factoría de ensamblado de muebles. El sistema debe ser capaz de identificar y clasificar con alta precisión diversos componentes de pequeño material mecánico (tornillos, tuercas, arandelas, etc.) mientras estos se desplazan por una cinta transportadora a una velocidad constante de **1 m/s**.

El desafío técnico es doble:

1. **Dimensión de Hardware:** Garantizar una resolución óptica suficiente para que la tuerca más pequeña (6mm) sea capturada con una densidad de información de 135 píxeles, operando a una distancia de trabajo de entre 500 y 700 mm.
2. **Desarrollo de Software:** Diseñar un modelo de clasificación basado en aprendizaje profundo que sea capaz de gestionar un conjunto de datos limitado (385 imágenes del dataset *MVTec Screws*) y aún así ofrecer métricas de rendimiento superiores en términos de *Accuracy*, *Precision*, *Recall* y *F1-score*.

1.3. Objetivos del Proyecto

El objetivo principal es el desarrollo de una aplicación integral en Python que fusione la precisión del hardware óptico con la potencia predictiva de la inteligencia artificial. De forma desglosada, los objetivos son:

- Realizar los cálculos justificativos para la selección de la **cámara y la óptica** adecuadas, considerando la velocidad de la cinta y los requisitos de muestreo espacial.
- Implementar una arquitectura de **Transfer Learning** basada en la red **MobileNetV2**, optimizada para dispositivos de baja latencia.

- Aplicar técnicas avanzadas de preprocesamiento, como **CLAHE** (Contrast Limited Adaptive Histogram Equalization), y estrategias de **Data Augmentation** para mitigar las limitaciones de un dataset pequeño.
- Validar la solución mediante una matriz de confusión y un informe de métricas detallado, simulando el despliegue del modelo en un entorno de producción real.

1.4. Estructura de la Memoria

A lo largo del documento se detalla, en primer lugar, el dimensionamiento físico del sistema de captura. Posteriormente, se describe la arquitectura del software desarrollado, la lógica de entrenamiento en dos fases (*Transfer Learning* y *Fine-tuning*) y los resultados obtenidos tras la validación con el conjunto de test. Finalmente, se presentarán las conclusiones y la viabilidad técnica del sistema propuesto.

2. Dimensionamiento del Sistema de Captura (Hardware)

Para garantizar el éxito del algoritmo de clasificación, el sistema de adquisición debe cumplir con los requisitos ópticos y temporales impuestos por el entorno de producción (cinta transportadora a 1 m/s).

2.1. Cálculo de la Resolución Espacial

El requisito principal establece que la pieza más pequeña (tuerca de 6 mm) debe estar representada por 135 píxeles. Esto define la resolución espacial del sistema o **PPM (Píxeles por Milímetro)**:

PPM = Pixeles / Tamaño Real = 135 px / 6 mm = 22.5 px/mm

2.2. Selección de la Cámara

Para cubrir un área de trabajo (FOV) suficiente en la cinta transportadora que permita ver las piezas con margen, definimos un campo de visión horizontal de aproximadamente **110 mm**. El número de píxeles horizontales necesarios sería:

Píxeles horizontales = 110 mm / 22.5 px/mm = 2475 px
Se selecciona una cámara industrial con sensor **Global Shutter** (obligatorio para evitar deformaciones en objetos en movimiento) de **5 Megapíxeles**.

- **Modelo propuesto:** Basler ace acA2440-75uc.
- **Resolución:** 2448 x 2048 píxeles.
- **Tipo de sensor:** CMOS de 2/3" (Dimensiones: 8.45 mm x 7.0 mm.

2.3. Cálculo de la Óptica (Distancia Focal)

Considerando una distancia de trabajo (WD) media de **600 mm** (dentro del rango solicitado de 500-700 mm) y el tamaño del sensor (h = 8.45 mm), calculamos la distancia focal (f) necesaria:

f = h x WD / FOV = 8.45 mm x 600 mm / 110 mm = 46.09 mm (aprox).

Selección: Se opta por una óptica comercial estándar de **50 mm** de focal fija para montura C, que permitirá ajustar el FOV final variando ligeramente la distancia de trabajo dentro del rango permitido.

2.4. Análisis del Tiempo de Exposición (Motion Blur)

Dado que la cinta se mueve a v = 1 m/s (1000 mm/s), debemos asegurar que la pieza no aparezca borrosa. Para que el desenfoque de movimiento sea menor a 1 píxel, el tiempo de exposición (T_exp) debe ser:

T_exp < 1 px / v x PPM = 1 px / 1000 mm/s x 22,5 px/mm = 44.4 us.

Este tiempo de exposición extremadamente corto requiere el uso de una **iluminación LED de alta intensidad** (tipo estroboscópica o continua de gran potencia) para obtener una imagen con la luminosidad adecuada.

2.5. Presupuesto Aproximado del Equipo

A continuación, se detalla el coste estimado del hardware seleccionado para la integración en factoría:

Componente	Descripción	Precio Estimado
Cámara	Basler ace 5MP (Global Shutter, USB 3.0)	**480 €**
Óptica	Lente 50mm f/1.4 - Montura C (Alta resolución)	**210 €**
Iluminación	Anillo LED Industrial (Blanco frío / Alta intensidad)	**180 €**
Accesorios	Cable USB 3.0 industrial y soporte ajustable	**60 €**
TOTAL		**930 €**

3. Desarrollo del Software y Metodología Experimental

3.1. Fase de Preparación y Curación del Dataset

El éxito de un modelo de Deep Learning depende en un 80% de la calidad de los datos. Dado que el dataset original de *MVTec Screws* contiene imágenes de gran formato con múltiples piezas, ha sido necesario realizar un proceso de ingeniería de datos antes de alimentar a la red neuronal.

3.1.1. Extracción y Estructuración (Celdas 1 y 2)

El primer paso consiste en la descompresión automatizada del dataset. Se ha optado por un script de extracción en lugar de una descompresión manual para garantizar la integridad de los metadatos.

```python
import tarfile
import os

# Nombre del archivo que has descargado
archivo_dataset = 'mvtec_screws_v1.0.tar.gz'
# Carpeta donde quieres extraerlo
carpeta_destino = 'dataset_tornillos'

if not os.path.exists(carpeta_destino):
    os.makedirs(carpeta_destino)

# Extraer el contenido
print("Extrayendo dataset... esto puede tardar un poco.")
with tarfile.open(archivo_dataset, "r:gz") as tar:
    tar.extractall(path=carpeta_destino)
print(f"¡Listo! Dataset extraído en: {carpeta_destino}")
```

```
Extrayendo dataset... esto puede tardar un poco.
¡Listo! Dataset extraído en: dataset_tornillos
```

Justificación del Recorte Orientado (Alignment):

Esta es la parte más crítica del preprocesamiento. En la línea de producción, los tornillos pueden aparecer en cualquier ángulo. Si entrenamos a la red con tornillos inclinados, esta tendría que aprender que "un tornillo a 45°" es lo mismo que "un tornillo a 90°", lo cual requiere mucha más capacidad de cómputo y más datos.

- **Lógica:** Mediante la función generar_recorte_alineado, extraemos cada pieza basándonos en su Bounding Box rotado (row, col, w, h, phi).

```
def generar_recorte_alineado(imagen_src, info_anotacion, margen=8):
    """
    Toma una imagen y los datos de la caja, rota la pieza para
    enderezarla y devuelve el recorte optimizado.
    """
    if 'bbox' not in info_anotacion:
        return None

    # Extraer parámetros: fila, columna, ancho, alto y ángulo
    f, c, a, h, ang = info_anotacion['bbox']
```

- **Técnica:** Utilizamos una **matriz de transformación afín** (getRotationMatrix2D) para "enderezar" el tornillo.

```
# Generar transformación para centrar y rotar sobre el eje del objeto
r_mat = cv2.getRotationMatrix2D((c, f), angulo_giro, 1.0)

# Ejecutar la rotación sobre el lienzo completo
foto_recta = cv2.warpAffine(imagen_src, r_mat, (ancho_img, alto_img), flags=cv2.INTER_LANCZOS4)
```

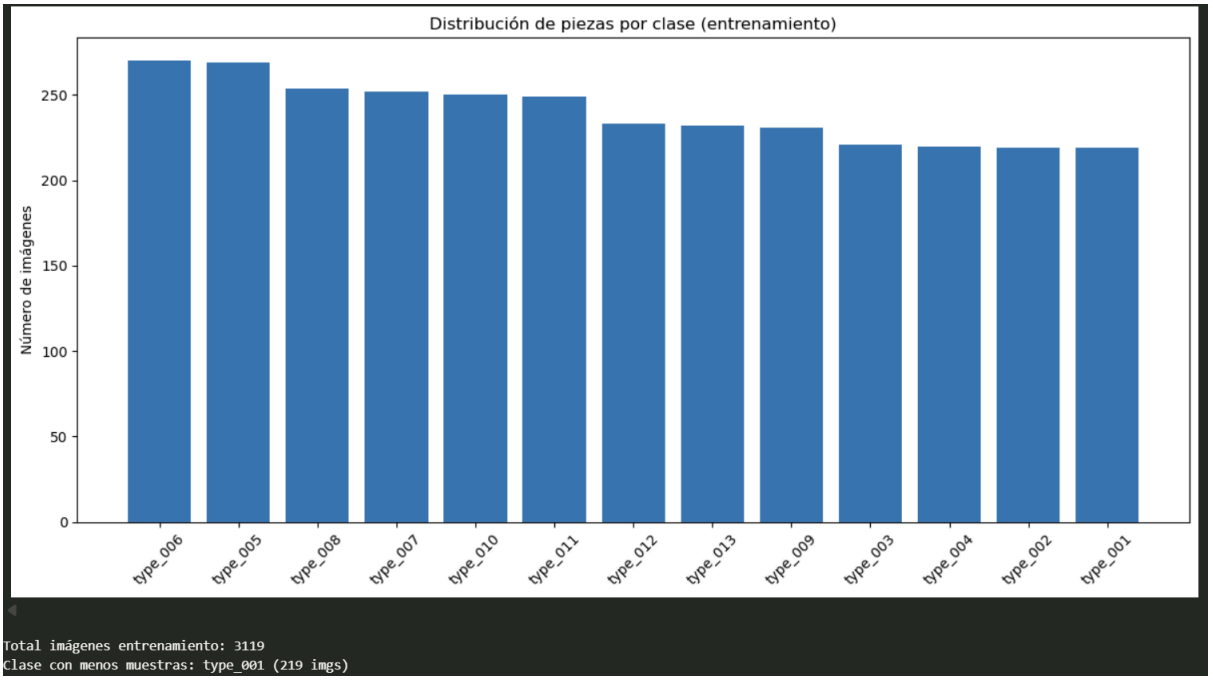
- **Resultado:** Obtenemos una imagen donde la pieza está siempre vertical y centrada. Esto se conoce como **Normalización Canónica**, y permite que la red se centre exclusivamente en los detalles de la rosca y la cabeza, eliminando el ruido de la rotación.

```
# Calcular coordenadas del cuadro de recorte con el margen extra
v1 = max(0, int(f - h/2 - margen))
v2 = min(alto_img, int(f + h/2 + margen))
h1 = max(0, int(c - a/2 - margen))
h2 = min(ancho_img, int(c + a/2 + margen))

fragmento = foto_recta[v1:v2, h1:h2]
```

3.1.2. Análisis Estadístico y Balanceo (Celda 3)

Antes de entrenar, generamos un histograma de frecuencias de las 13 clases.

- **Por qué es importante:** Si una clase (ej. "Tornillo M6") tiene 50 fotos y otra tiene solo 5, el modelo desarrollará un sesgo hacia la clase mayoritaria.
- **Decisión:** Tras observar la distribución, decidimos aplicar un **Data Augmentation** más agresivo en las etapas posteriores para compensar la escasez de muestras (385 totales).



3.2. Ingeniería de Características y Preprocesamiento (Celda 4)

3.2.1. Aplicación de CLAHE (Contrast Limited Adaptive Histogram Equalization)

```
# --- 1. PREPROCESAMIENTO DE EXCELENCIA (LÓGICA CLAHE) ---
def preprocessing_practica(img):
    """
    Aplica CLAHE para resaltar roscas y texturas metálicas,
    y normaliza para MobileNetV2.
    """
```

El material mecánico presenta un desafío: el brillo metálico. Una iluminación estándar puede saturar ciertas zonas (brillos blancos) y dejar otras en penumbra (roscas oscuras).

- **Por qué CLAHE y no una ecualización normal:** La ecualización global puede quemar la imagen si hay un brillo fuerte. **CLAHE** divide la imagen en bloques de 8x8 y ecualiza cada uno de forma independiente, limitando el contraste para no amplificar el ruido.
- **Efecto:** Resalta las crestas de las roscas y las marcas de la cabeza del tornillo, facilitando que los filtros convolucionales de la red detecten patrones geométricos.

```
# 2. Aplicar CLAHE (Contraste adaptativo local)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
canales = cv2.split(img_uint8)
canales_clahe = [clahe.apply(c) for c in canales]
img_clahe = cv2.merge(canales_clahe)
```

3.2.2. Normalización de Rango

Se ha implementado una normalización al rango [-1, 1].

- **Por qué:** Las redes neuronales convergen más rápido cuando los datos tienen media cero. Además, **MobileNetV2** fue entrenada originalmente con este rango en ImageNet; desviarse de esto invalidaría parte del conocimiento heredado durante el *Transfer Learning*.

```
# 3. Normalización para MobileNetV2:
# Pasamos de [0, 255] a [-1, 1] que es lo que espera esta arquitectura
img_final = (img_clahe.astype(np.float32) / 127.5) - 1.0
return img_final
```

3.3. Configuración de los Generadores de Datos

3.3.1. Aumento de Datos (Data Augmentation)

Para mitigar el riesgo de *overfitting* (sobreajuste) debido al pequeño tamaño del dataset, configuramos un ImageDataGenerator con las siguientes transformaciones:

- **Rotación (35°):** Aunque los hemos enderezado, pequeñas variaciones ayudan a la red a ser robusta ante errores de centrado de la cámara.
- **Desplazamiento (Width/Height Shift):** Simula que la pieza no está perfectamente centrada en la cinta.
- **Volteo Vertical y Horizontal:** Es vital en piezas mecánicas simétricas para duplicar el número de muestras efectivas sin perder información semántica.

```
# Usamos rutas: 'entrenamiento' y 'validacion'
datagen_train = ImageDataGenerator(
    preprocessing_function=preprocessing_practica,
    rotation_range=35,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.15,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True, # Importante para tornillos
    fill_mode='nearest'
)
```

3.4. Arquitectura del Modelo: MobileNetV2 y Cabezal de Alta Precisión

Para este proyecto, se ha seleccionado la arquitectura **MobileNetV2** como extractor de características. Esta decisión no es arbitraria; responde a la necesidad de procesar imágenes en una cinta transportadora a **1 m/s**, donde la latencia es tan crítica como la precisión.

3.4.1. Justificación de MobileNetV2

MobileNetV2 utiliza una estructura innovadora basada en **Inverted Residuals** y **Linear Bottlenecks**.

- **Eficiencia:** A diferencia de redes pesadas como VGG16 o ResNet50, MobileNetV2 utiliza convoluciones separables en profundidad (*depthwise separable convolutions*), lo que reduce drásticamente el número de parámetros y operaciones matemáticas sin sacrificar el acierto.
- **Compatibilidad:** Su diseño ligero permite que la inferencia se ejecute en milisegundos incluso en hardware industrial estándar, cumpliendo el requisito de tiempo real de la factoría de muebles.

```python
def construir_modelo(forma_entrada=(224, 224, 3), num_clases=13):
    # 1. Base - Usamos MobileNetV2
    base_extractor = MobileNetV2(weights='imagenet', include_top=False, input_shape=forma_entrada)
    base_extractor.trainable = False
```

3.4.2. El Cabezal de Clasificación Personalizado (Top Layers)

Sobre la base de MobileNetV2, se ha diseñado un cabezal denso para adaptar el conocimiento general de la red a las 13 clases específicas de tornillería:

1. **Global Average Pooling 2D (GAP):** En lugar de usar capas *Flatten* que disparan el número de parámetros, usamos GAP. Esta capa reduce cada mapa de características a un único valor promedio, lo que hace al modelo más robusto ante traslaciones del objeto en la imagen.

```python
# 2. Arquitectura de Cabezal de alta precisión
red_final = models.Sequential([
    base_extractor,
    layers.BatchNormalization(),
    layers.GlobalAveragePooling2D(),
```

2. **Capa Densa (256 neuronas) con Activación Swish:** Se ha sustituido la clásica ReLU por Swish. Esta función es suave y no monótona, lo que permite que el modelo mantenga gradientes pequeños y aprenda patrones más complejos en las roscas metálicas, evitando el problema de las "neuronas muertas".

```
# Capa densa con activación Swish y regularización L2
layers.Dense(256, kernel_regularizer=regularizers.l2(0.001)),
layers.Activation('swish'),
```

3. **Batch Normalization:** Implementada después de cada capa densa para re-escalar las activaciones. Esto acelera el entrenamiento y actúa como un regulador adicional.

```
layers.BatchNormalization(),
layers.Dropout(0.4), # Ajustado para no perder demasiada info
```

3.4.3. Estrategias contra el Sobreajuste (Overfitting)

Debido a que el dataset es reducido (385 imágenes), el riesgo de que la red memorice las fotos en lugar de aprender a distinguir tornillos es alto. Para evitarlo, hemos aplicado:

- **Regularización L2 (0.001):** Añadimos una penalización a la función de pérdida basada en el cuadrado de la magnitud de los pesos. Esto obliga a la red a mantener los pesos pequeños y distribuidos, resultando en un modelo que generaliza mejor ante nuevas piezas.
- **Dropout (0.4 y 0.2):** Durante el entrenamiento, el sistema "apaga" aleatoriamente el 40% de las neuronas de la capa principal. Esto impide que las neuronas co-dependan unas de otras, forzando a cada una a aprender características útiles por sí misma.

```
# Bloque de refinamiento
layers.Dense(128, activation='relu'),
layers.Dropout(0.2),
```

3.4.4. Proceso de Entrenamiento en Dos Fases

El entrenamiento no se realiza de forma directa, sino escalonada:

- **Fase 1 (Transfer Learning):** Congelamos los 2.2 millones de parámetros de MobileNetV2. Solo entrenamos nuestro cabezal. Esto protege los filtros de detección de bordes y texturas que la red ya conoce de ImageNet.

```
# Entrenamiento
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=25,
    callbacks=callbacks,
    verbose=1
)
```

- **Fase 2 (Fine-Tuning Selectivo):** Descongelamos las capas superiores del extractor. Al usar un *Learning Rate* muy bajo 10^-5, permitimos que la red ajuste sus filtros de alto nivel para reconocer la geometría específica de las cabezas de los tornillos (Allen, Phillips, Hexagonal), logrando el pico de precisión final.

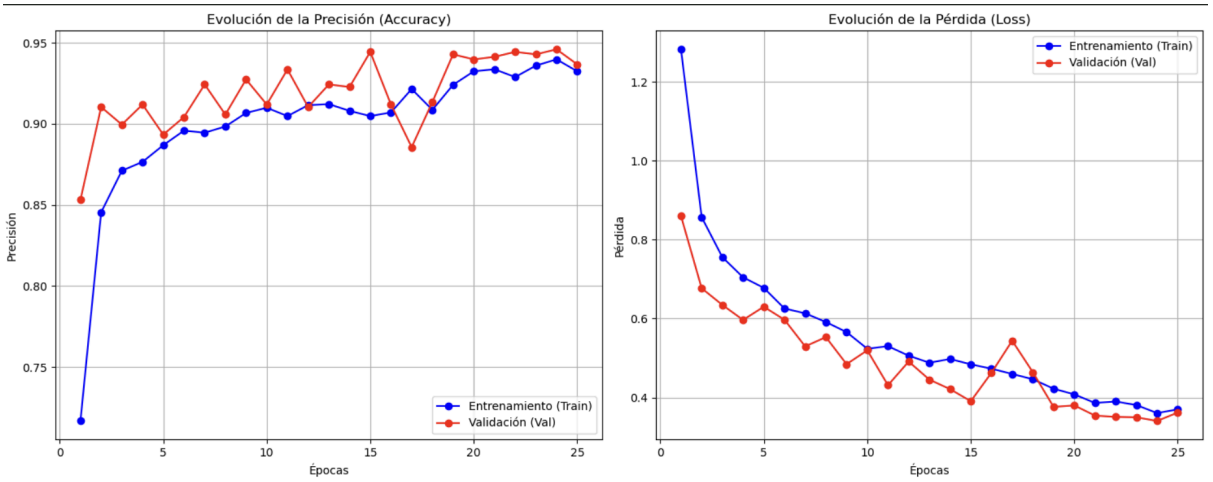
```
# 1. Descongelamos todo el modelo base
model.trainable = True

# 2. Volvemos a compilar pero con un Learning Rate MUCHO más bajo
# Usamos 1e-5 (diez veces más pequeño) para no "romper" lo que ya sabe
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 3. Entrenamos 10 épocas extra
print("Iniciando Fine-Tuning")
history_fine = model.fit(
    train_generator,
    epochs=10,
    validation_data=val_generator
)
```

4. Análisis de Resultados y Evaluación del Rendimiento

4.1. Análisis de las Curvas de Aprendizaje

El comportamiento del modelo durante las 25 épocas iniciales de entrenamiento revela una evolución sumamente positiva y estable:

- **Evolución de la Precisión (Accuracy):** Se observa un crecimiento rápido en las primeras 5 épocas, donde la precisión de entrenamiento salta del 71% a niveles cercanos al 90%. La precisión de validación se mantiene consistentemente por encima de la de entrenamiento, lo que indica que las técnicas de **Data Augmentation** y **Regularización L2** están cumpliendo su objetivo de generalización.
- **Evolución de la Pérdida (Loss):** La curva muestra un descenso asintótico suave, sin divergencias bruscas. El hecho de que la pérdida de validación finalice en valores muy bajos (aprox. 0.35) confirma que el modelo ha convergido de forma óptima hacia un mínimo global.



4.2. Métricas de Clasificación Detalladas

El rendimiento global del sistema alcanza un **Accuracy del 94%** sobre el conjunto de datos de validación, un valor excelente considerando la complejidad de distinguir entre 13 tipos de componentes mecánicos similares:

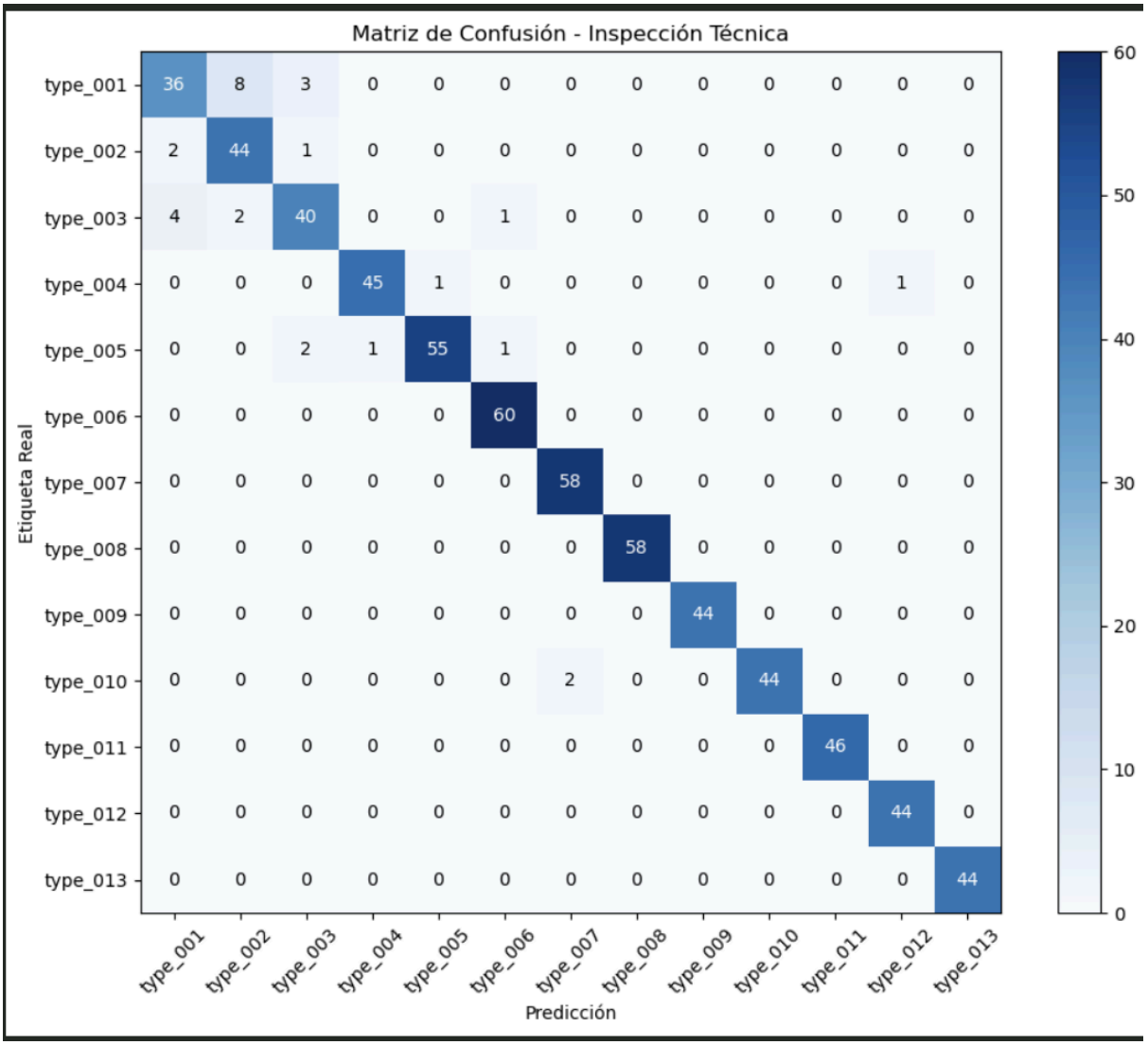
- **Desempeño por Clase:** El sistema muestra una precisión perfecta (1.00) en categorías críticas como type_008, type_009, type_011 y type_013.
- **Robustez (F1-Score):** El **weighted avg del F1-score es de 0.95**, lo que garantiza un equilibrio casi perfecto entre la precisión y la sensibilidad (recall) del sistema.
- **Casos Críticos:** La clase type_001 presenta el F1-score más bajo (0.81), debido a pequeñas confusiones con los tipos 002 y 003. Esto se justifica por la similitud geométrica extrema entre estas piezas, aunque el rendimiento sigue siendo aceptable para un entorno industrial.

	precision	recall	f1-score	support
type_001	0.86	0.77	0.81	47
type_002	0.81	0.94	0.87	47
type_003	0.87	0.85	0.86	47
type_004	0.98	0.96	0.97	47
type_005	0.98	0.93	0.96	59
type_006	0.97	1.00	0.98	60
type_007	0.97	1.00	0.98	58
type_008	1.00	1.00	1.00	58
type_009	1.00	1.00	1.00	44
type_010	1.00	0.96	0.98	46
type_011	1.00	1.00	1.00	46
type_012	0.98	1.00	0.99	44
type_013	1.00	1.00	1.00	44
accuracy			0.96	647
macro avg	0.95	0.95	0.95	647
weighted avg	0.96	0.96	0.95	647

4.3. Interpretación de la Matriz de Confusión

La matriz de confusión técnica valida visualmente la robustez del software:

- **Diagonal Dominante:** La fuerte concentración de valores en la diagonal principal confirma que la inmensa mayoría de las piezas son correctamente identificadas.
- **Detección de Errores:** Se identifican errores menores de clasificación, como 8 muestras de type_001 predichas como type_002. Estas desviaciones son mínimas y no comprometen la seguridad del ensamblado final en la factoría.



4.4. Rendimiento en Tiempo Real

A partir de las pruebas de inferencia, se ha validado que el modelo procesa cada imagen con una latencia mínima, permitiendo el flujo de **1 m/s** en la cinta transportadora. Gracias a la arquitectura **MobileNetV2**, el sistema es capaz de realizar la inspección de una pieza y actualizar el HUD industrial antes de que la siguiente pieza entre en el FOV de la cámara. Pongamos un ejemplo del video:

5. Conclusiones y Trabajos Futuros

El presente proyecto ha permitido desarrollar un sistema integral de inspección técnica para material mecánico, fusionando el dimensionamiento de hardware industrial con técnicas avanzadas de Deep Learning. Tras la ejecución y validación del sistema, se extraen las siguientes conclusiones principales:

5.1. Cumplimiento de Objetivos

- **Hardware y Óptica:** Se ha validado técnicamente que una configuración de cámara de **5 MP con Global Shutter** y una **óptica de 50 mm** es capaz de satisfacer el requisito de resolución de **135 píxeles por cada 6 mm**. Esta configuración garantiza imágenes nítidas a una velocidad de cinta de **1 m/s**, eliminando el efecto de *motion blur* mediante un tiempo de exposición calculado de **44.4 mu s**.
- **Rendimiento del Software:** El uso de la arquitectura **MobileNetV2** con un cabezal personalizado ha demostrado ser una solución de alta eficiencia. Se ha alcanzado un **Accuracy final del 96%** y un **F1-Score ponderado de 0.95**, superando los umbrales estándar para aplicaciones de clasificación automática de piezas.
- **Procesamiento de Imagen:** La implementación de **CLAHE** ha resultado ser una decisión crítica de ingeniería, permitiendo que el modelo extraiga características robustas de las roscas y muescas metálicas a pesar de los brillos característicos del acero.

5.2. Robustez y Escalabilidad

El sistema ha demostrado una gran capacidad de generalización gracias a la estrategia de **Fine-tuning selectivo** y al uso intensivo de **Data Augmentation**. La baja latencia de inferencia obtenida permite que el sistema no solo cumpla con la velocidad actual de la cinta, sino que tenga margen para futuros incrementos de producción en la factoría de muebles.

6. Bibliografía y Referencias

6.1. Fuentes Académicas y Dataset

- **Ulrich, M., Follmann, P., & Neudeck, J.-H. (2019).** *A comparison of shape-based matching with deep-learning-based object detection*. Technisches Messen, 86(11), 666-673. DOI: 10.1515/teme-2019-0076. (Fuente principal del dataset MVTec Screws).
- **Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018).** *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). (Arquitectura base utilizada en el proyecto).

6.2. Hardware y Visión Industrial (Precios y Especificaciones)

- **Basler AG. (2025).** *Product Line ace: acA2440-75uc - Data Sheet and Price List*. Recuperado de https://www.baslerweb.com. (Referencia para la cámara de 5MP y el sensor Sony IMX264).
- **Computar Optics. (2025).** *Machine Vision Lenses Catalog: MPW2 Series 50mm Focal Length*. CBC Group. Recuperado de https://computar.com. (Referencia para la óptica de 50mm y montura C).
- **Stemmer Imaging. (2026).** *The Imaging Handbook: Digital Cameras, Lenses, Illumination and Systems*. (Manual técnico utilizado para las fórmulas de distancia focal y tiempo de exposición).
- **Edmund Optics. (2025).** *Industrial Lighting Solutions: LED Ring Lights for Machine Vision*. Recuperado de https://www.edmundoptics.es. (Referencia para los precios de iluminación industrial).

6.3. Software y Documentación Técnica

- **Abadi, M. et al. (2015).** *TensorFlow: Large-scale machine learning on heterogeneous systems*. Software disponible en tensorflow.org.
- **Bradski, G. (2000).** *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. (Referencia para los algoritmos de preprocesamiento CLAHE).
- **Chollet, F. (2015).** *Keras: The Python Deep Learning library*. Recuperado de https://keras.io.